Les instructions à choix multiples match-case

From version 3.10 upwards

```
lang = input("What's the programming language you want to learn? ")
match lang:
    case "JavaScript":
        print("You can become a web developer.")
    case "Python":
        print("You can become a Data Scientist")
    case "PHP":
        print("You can become a backend developer")
    case "Solidity":
        print("You can become a Blockchain developer")
    case "Java":
        print("You can become a mobile app developer")
    case _:
        print("The language doesn't matter, what matters is solving problems.")
```

```
Python 3.9.13 (main, Aug 25 2022, 23:51:50) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.31.1 -- An enhanced Interactive Python.

In [1]: runfile('C:/Users/TEK-UP/Desktop/TEK-UP 2022/Semestre 2 2022_2023/Python/
wdir='C:/Users/TEK-UP/Desktop/TEK-UP 2022/Semestre 2 2022_2023/Python/Cours 2023/
  File "C:\Users\TEK-UP\Desktop\TEK-UP 2022\Semestre 2 2022_2023\Python\Cours 20
    match lang:
          ^
SyntaxError: invalid syntax
```

```
C:\Users\TEK-UP\PycharmProjects\pythonProject\venv\Scripts\python.exe C:\Use
What's the programming language you want to learn? Python
You can become a Data Scientist

Process finished with exit code 0
```

```
C:\Users\TEK-UP\PycharmProjects\pythonProject\venv\Scripts\python
What's the programming language you want to learn? c language
The language doesn't matter, what matters is solving problems.

Process finished with exit code 0
```

Les fonctions: Définition & Appel

Définition d'une fonction

Objectif : structurer un programme en des sous programmes.

Avantages :

- Meilleure organisation du programme (regrouper les tâches par blocs : lisibilité et facilité de maintenance).
- Eviter la redondance et assurer une meilleure réutilisation du code.
- Possibilité de partager les fonctions (via des modules)

Définition d'une fonction

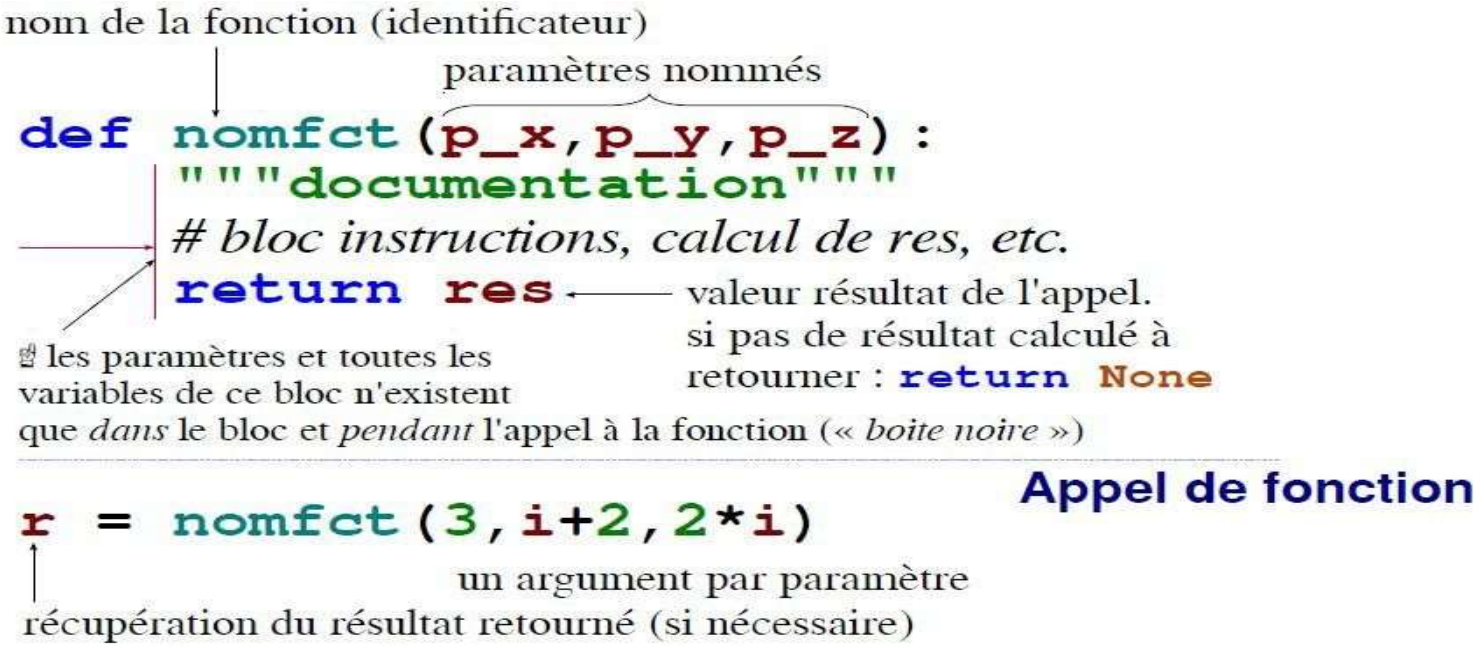
Définition : Une fonction est:

- Un groupe d'instructions regroupées sous un nom et s'exécutant à l'appel.
- Peut prendre des paramètres en entrée (non typés).
- Renvoie une valeur en sortie (ou plusieurs valeurs, ou pas de valeurs du tout : procédure)

Syntaxe :

```
def nom_fonction(liste de paramètres):
    bloc d'instructions
```

Définition d'une fonction



Exemple

```
""" on va faire un programme qui  calcule la division de 2 entiers  """
def division(a,b):
    if(b!=0):
        return a/b
    else:
        return None

a=int(input("Entrez un entier a = "))
b=int(input("Entrez un entier b = "))
d=division(a,b)
if(d!=None):
    print("Résultat de division : ", d)
else:
    print("Erreur ! Division par zéro")
```

Des observations ?

- **Indentation** remplace les {} du C/Java/...
- Typage dynamique

```
In [32]: runfile('C:/Users/Sawssen/python codes/s2.p
Users/Sawssen/python codes')

Entrez un entier a = 5

Entrez un entier b = 2
Résultat de division :  2.5
```

Appel de fonction

mode de passage de paramètres

Les règles de passage des paramètres sont les suivantes :

- Les paramètres **immutables (non modifiables)** correspondent au **passage par valeur**. En fait le passage se fait bien par référence, mais puisqu'il est impossible de changer des objets non modifiables, cela revient à faire une copie des données.

- Les paramètres **mutables**, tels les listes, les dictionnaires… correspondent au **passage par référence** (par adresse: **VAR**, **S** et **ES** en algorithmique).

- Dans n'importe quelle fonction on peut inverser les arguments lors d'un appel si on les nomme :

```
1  def puissance(a, b):
2      return a**b
3
4  puissance(2,3)==puissance(b=3,a=2)
```

Portée de variables: locales Vs globales

- Les variables définies localement dans les fonctions sont uniquement visibles dans ces fonctions.

- Les variables définies en dehors de la fonction (dans la mémoire globale) sont accessibles dans la fonction. Mais ne peuvent pas changer de valeur (sauf si on les précède du mot clé global).

```
x=5
def addx(y):
    x=x+y
    return(x)

print(addx(10))
```

```
File "C:\Users\Sawssen\python codes\s2.py", line 72, in <module>
  print(addx(10))

File "C:\Users\Sawssen\python codes\s2.py", line 69, in addx
  x=x+y

UnboundLocalError: local variable 'x' referenced before assignment
```

```
x=5
def addx(y):
    global x
    x=x+y
    return(x)

print(addx(10))
```

```
In [55]: runfile('C:/Users/Sawssen/python codes/
Users/Sawssen/python codes')
15
```

Fonctions avec un nombre variable d'arguments

La fonction **`print`** peut prendre un nombre arbitraire d'arguments. De la même manière on peut definir des fonctions avec un nombre arbitraire d'arguments :

```
1  def add(a,b, *args):
2      s=a+b
3      for n in args:
4          s+=n
5      return s
```

Les arguments sont stockés sous forme de liste. Si l'on veut transformer une liste en arguments d'une fonction :

```
6  l=[2,3,4,5,6]
7  add(*l) # étoile "déliste" l en arguments
```

On ne peut utiliser `*` de cette manière que dans un appel ou une déclaration de fonction.

Fonctions avec un nombre variable d'arguments

On peut également avoir un nombre variable d'arguments nommés, utile quand on a beaucoup d'arguments optionnels ayant une signification différente :

```
1  def exemple(**kwargs):
2      for k in kwargs:
3          print("arg\"", k," \"=", kwargs[k])
```

De la même manière, on peut utiliser `**` pour "dédictionnariser" un dictionnaire en arguments de fonction.

```
4  exemple(**{"x":4,"h":"hache"})
5  exemple(x=4,h="hache")
```

Fonction anonyme Lambda

•Soit à définir en python la

fonction f : x → x^2

```
def f(x):
        return x**2
```

- Une fonction lambda (lettre grecque) est une façon simple de définir une fonction en Python.
- Une alternative condensée de f(x) est :

```
f = lambda x: x**2
```

```
def f(x):
    return x**2
print("Version def",f(2))

f = lambda x: x**2
print("Version Lambda",f(2))
```

Fonction anonyme Lambda

- Les Fonctions lambda sont aussi appelés Fonctions anonymes, Expressions lambda ou encore Abstractions lambda.

- Ce sont des fonctions dont le code correspond à **une seule expression**.

- Elles ne sont pas restreintes au entiers et aux réels, elles peuvent prendre comme paramètre un objet quelconque.

```
def longueur(f,s):
    return f(s)
print(longueur(lambda x:len(x), 'ticA'))
```

```
In [204]: runfile('C:/Users/Sawsse
4
```

Fonction anonyme Lambda

- Lors de l'utilisation du mode interactif, l'appel de la fonction lambda se fait à travers le tiret bas _

```
In [205]: lambda x,y: x+y
Out[205]: <function __main__.<lambda>(x, y)>

In [206]: _(3,4)
Out[206]: 7
```

- On peut définir et appeler en même temps une fonction anonyme

```
In [207]: print((lambda x,y:x+y)(3,7))
10
```

- La fonction anonyme Lambda. Elle permet de faire appel à une fonction non encore définie, qui sera définie et paramétrée lors de son appel

Fonctions Python intégrées

Built-in Functions

A
abs()
aiter()
all()
any()
anext()
ascii()

B
bin()
bool()
breakpoint()
bytearray()
bytes()

C
callable()
chr()
classmethod()
compile()
complex()

D
delattr()
dict()
dir()
divmod()

E
enumerate()
eval()
exec()

F
filter()
float()
format()
frozenset()

G
getattr()
globals()

H
hasattr()
hash()
help()
hex()

I
id()
input()
int()
isinstance()
issubclass()
iter()

L
len()
list()
locals()

M
map()
max()
memoryview()
min()

N
next()

O
object()
oct()
open()
ord()

P
pow()
print()
property()

R
range()
repr()
reversed()
round()

S
set()
setattr()
slice()
sorted()
staticmethod()
str()
sum()
super()

T
tuple()
type()

V
vars()

Z
zip()

_
__import__()

N. B : Pour plus de détails sur ces fonctions visitez l'url : https://docs.python.org/3/library/functions.html

Les exceptions

La gestion des erreurs (les exceptions)

Même si une instruction ou une expression est syntaxiquement correcte, elle peut générer une erreur lors de son exécution.

```
x=0
y=10
print(y/x)
```

```
ZeroDivisionError: division by zero
```

```
f=open("fichierInexistant.txr","r")
```

```
FileNotFoundError: [Errno 2] No such file or directory: 'fichierInexistant.txr'
```

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't convert 'int' object to str implicitly
```

La gestion des erreurs (les exceptions)

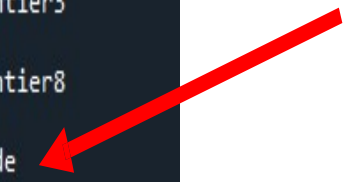
Soit le code suivant:

- Exécution sans exception

```
x = int(input("Donner un entier"))
y  =int(input("Donner un entier"))
print(x/y)
print("Suite de code")
```

```
Donner un entier3

Donner un entier8
0.375
Suite de code
```



```
Donnez un entier2

Donnez un entier0
Traceback (most recent call last):

  File "C:\Users\Sawssen\python codes\CoursFiles\untitled0.py", line 10, in <module>
    print(x/y)

ZeroDivisionError: division by zero
```

La gestion des erreurs (les exceptions)

- La partie du code susceptible de déclencher une exception est mise dans un bloc **try**
- Le gestion de l'exception est mise dans un bloc **expect**

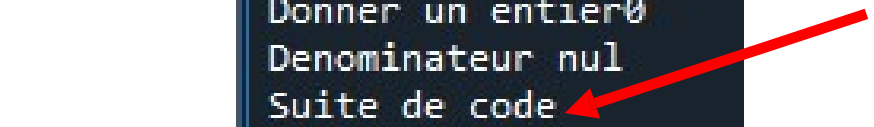
```
x = int(input("Donner un entier"))
y  =int(input("Donner un entier"))
try:
    print(x/y)

except:
    # gestion de l'exception
    print("Denominateur nul")
    y=1
    '''
     un exemple de traitement à faire
     si l'ecxeption est soulvée
    '''

print("Suite de code")
```

```
Donner un entier 5

Donner un entier0
Denominateur nul
Suite de code
```



- On note que le programme continu à s'exécuter même en présence de l'exception

Structure générale d'un programme

Strcture générale d'un programme Python

Chifoumi.py

```
import random
...
from random import *
from random import randint
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
def ecrire( nbre ):
    ...
def augmenter_score( mon_coup, ton_coup):
    ...
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
ton_score=0
mon_score=0
fin=5
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
print("Pierre-papier-ciseaux. Le premier à",fin,"a gagné !")
no_manche = 0
while mon_score < fin and ton_score < fin:
    ton_coup = int (input("1 : pierre, 2 : papier, 3 : ciseaux ? "))
    while ton_coup < 1 or ton_coup > 3:
        ton_coup =int (input("1 : pierre, 2 : papier, 3 : ciseaux ? "))
```

→ Import des modules

→ Définition des fonctions

→ Initialisation de l'algorithme

→ Algorithme du jeu (itératif)